2026-11-05 넥스트에 적용된 구조

- 날짜: 2026-11-05
- 앱: `lgcns-fe-next`
- 패키지: `next` 16.4.3, `react` 19.3.0, `next-auth` 4.24.15, `prisma` 6.19.3, `jose` 6.2.15, `quill` 2.0.3, `sanitize-html` 2.18.1
- 기준: 이 날짜의 작업 트리에 있는 소스. 아래 설명은 그 파일이 실제로 호출하는 동작만 담는다.
- 공식 문서 조회: Next.js App Router, React 19, MDN, HTML 스크립트 로딩. 주소는 각 절에 적는다.

1. 파일이 맡은 일

화면 조합은 `component/`에 있고, 서버 변경은 `actions/`를 통한다. Prisma는 `lib/prisma.ts`와 `service/`에서만 가져온다.

계층	경로	이 코드가 하는 일
라우트	`app/`	URL, 레이아웃, Route Handler. 조회는 service를 감싼 `lib/postCache.ts` 또는 `readCurrentUser`를 부른다
Action	`actions/postActions.ts`, `actions/userActions.ts`	파일 첫 줄 `"use server"`. `FormData`를 읽고 service를 부른 뒤 `updateTag` 또는 `redirect`, 가입은 다음 단계 상태를 반환한다
Service	`service/postService.ts`, `service/userService.ts`	검증, 작성자 확인, Prisma. `redirect`, `cookies`, `FormData`를 쓰지 않는다
캐시	`lib/postCache.ts`	`"use cache"`로 목록과 글 하나를 감싼다. 쿠키는 읽지 않는다
세션 읽기	`lib/currentUser.ts`, `component/providers/AuthShell.tsx`	`connection()` 다음 쿠키 또는 `getServerSession`
요청 앞단	`lgcns-fe-next/proxy.ts`	렌더 전에 로그인 여부를 보고 리다이렉트한다
UI	`component/atoms`, `molecules`, `organisms`, `templates`	atom·molecule은 action 모듈을 import하지 않는다. organism이 `form action`으로 받는다

가입 화면은 `app/(auth)/signup/page.tsx`가 `submitSignup`을 `SignupSection`에 넘기고, 그 폼의 `action`에 연결한다. 글 카드 `PostCard`는 `"use client"`이고 `toggleLike`, `deletePost`, `createComment`를 `form action`으로 건다. service를 직접 부르지 않는다.

2. 선언 방식과 로딩 시점

이 앱의 페이지 파일에는 `<script>` 태그가 없다. 브라우저가 스크립트를 받는 방식은 파일 선언으로 갈린다.

MDN의 `script` 요소는 속성에 따라 파서를 막는 시점이 다르다.

- 속성 없는 고전 스크립트는 받는 동안 HTML 파서를 멈춘다. 동기 실행이다.

- async는 받는 즉시 실행하고, 문서 안 순서를 지키지 않는다.
- defer는 문서 파싱과 병렬로 받고, 파싱이 끝난 뒤 문서 순서대로 실행한다.
- type="module"은 기본이 defer와 같이 파싱을 막지 않는다. 모듈 그래프가 준비된 뒤 실행하고, 엄격 모드다. 출처: MDN script, HTML Living Standard의 script 요소.

import()는 그 표현식이 평가될 때 모듈을 가져오는 Promise다. 페이지를 열 때 자동으로 평가되지 않는다. 출처: MDN import().

코드	선언	실행 위치	브라우저가 JS를 받는 시점
app/(post)/posts/page.tsx	지시어 없음. 서버 컴포넌트	서버	이 함수 본문의 JS는 브라우저로 내려가지 않는다. 결과는 HTML이다
PostCard.tsx 첫 줄 "use client"	클라이언트 컴포넌트	서버에서 RSC 페이로드로 직렬화되고, 브라우저에서 하이드레이션	Next가 만드는 모듈 스크립트다. 페이지 소스에 async나 defer를 손으로 쓰지 않는다. type="module" 계열로 파서를 막지 않는다
PostEditor.tsx의 await import("quill")	useEffect 안의 동적 import()	브라우저, 마운트 후	첫 화면의 모듈 그래프 밖이다. effect가 돈 뒤에 별도 청크를 받는다
import "quill/dist/quill.snow.css"	클라이언트 파일의 CSS import	번들된 스타일	스크립트 실행이 아니다. Quill 테마 CSS가 그 청크와 함께 온다
app/layout.tsx의 next/font/google	Geist(), Gothic_A1(), Black_Han_Sans() 호출	빌드 시 폰트 CSS 변수	<script>가 아니다. 출처: next/font
actions/postActions.ts 첫 줄 "use server"	파일의 export가 모두 서버 함수	브라우저가 폼을 POST할 때 서버	함수 본문은 클라이언트 번들에 들어가지 않는다
lib/postCache.ts 함수 본문 "use cache"	캐시 지시어. 파일 전체가 아니다	캐시 미스일 때 서버에서 Prisma	브라우저 스크립트가 아니다
proxy.ts의 export async function proxy	Proxy 파일 규약	라우트 렌더 전 서버	페이지 JS보다 앞이다
app/api/auth/[...nextauth]/route.ts	GET, POST export	그 경로의 HTTP 메서드	페이지 렌더와 별개의 응답이다

Gothic_A1은 app/layout.tsx에서 변수를 만들지만, <html className>에는 Black_Han_Sans, Geist, Geist_Mono 변수만 붙는다. gothicA1.variable은 그 className에 없다.

app/(auth)/client-login/page.tsx는 파일 첫 줄이 "use client"인 페이지다. onSubmit에서 preventDefault를 하고 console.log만 한다. signIn이나 /api/auth/login을 부르지 않는다.

3. 라우트

라우트 그룹 폴더 (auth), (main), (post), (postsys)는 URL에 들어가지 않는다. 레이아웃만 나눈다. (main)/layout.tsx와 (post)/layout.tsx는 Header를 감싸고, (auth)/layout.tsx는 세로 flex만 준다. (auth)/signup/layout.tsx는 children을 그대로 반환한다.

URL	파일	params
/	app/(main)/page.tsx	없음
/feed	app/(main)/feed/page.tsx	searchParams Promise
/blog/[blogId]	app/(main)/blog/[blogId]/page.tsx	params: Promise<{ blogId: string }>
/product/...	app/(main)/product/[...productCategoryName]/page.tsx	catch-all 배열
/cart	app/(main)/cart/page.tsx	없음. 본문은 Cart
/login, /signup	app/(auth)/.../page.tsx	로그인 쿼리는 클라이언트 useSearchParams
/posts	app/(post)/posts/page.tsx	없음
/postdetail/[postId]	app/(post)/postdetail/[postId]/page.tsx	세그먼트 하나
/postwrite, /postwrite/[postId]	postwrite/[[...postId]]/page.tsx	선택적 catch-all

선택적 catch-all [[...postId]]는 세그먼트가 없어도 페이지가 맞는다. 코드는 세그먼트가 없으면 작성, 하나면 수정, 둘 이상이면 /posts로 redirect한다. 수정 전용 postmodify 페이지 파일은 없다.

Next.js 15부터 페이지의 params와 searchParams는 Promise다. 동기 속성으로 읽지 않는다. 출처: Upgrading to version 15, Async Request APIs.

피드, 블로그, 상품은 페이지가 Promise를 넘기고, Suspense 자식이 await 한다. FeedContent는 page, limit, sort, search를 await해서 화면에 찍는다. Prisma를 부르지 않는다. FeedNextButton은 "use client"이고 useRouter().push로 다음 page 쿼리를 만든다.

글 상세와 글쓰기는 페이지 함수가 먼저 await params를 하고, 그 다음 Suspense 안에서 readCurrentUser와 getCachedPost를 부른다. 제목과 목록 링크는 Suspense 밖에 있다.

app/loading.tsx는 SignupFallback을 반환한다. 글 목록의 "글 목록을 불러오는 중..."은 posts/page.tsx의 Suspense fallback이라서 loading.tsx와 별개다. 루트 레이아웃은 AuthShell을 <Suspense fallback={null}>로 감싼다. metadata.title은 Create Next App이다.

4. Cache Components

next.config.ts는 공식 시작 문서와 같은 두 플래그를 켠다. cacheComponents: true, partialPrefetching: true. 출처: Next.js Caching, cacheComponents. 데이터 조회는 기본이 동적이고, "use cache"를 붙인 함수나 컴포넌트만 캐시한다. partialPrefetching을 빠뜨리면 경고가 난다.

lib/postCache.ts의 두 함수는 본문 첫 지시어가 "use cache"다. 이어서 cacheTag("posts"), cacheLife("max")를 호출하고 postService.listPosts() 또는 getPost를 반환한다.

공식 cacheLife 프리셋 max는 잘 안 바뀌는 콘텐츠용이다. 출처: cacheLife.

값	의미	max
stale	클라이언트가 서버에 다시 묻지 않고 캐시를 보여주는 시간	5분
revalidate	서버가 다음 요청에서 백그라운드로 다시 만드는 간격	30일
expire	그 시간이 지나고 요청이 없으면 다음 요청이 새 결과를 기다림	1년

next.config.ts에는 cacheLife 재정의가 없다. 문서의 stale은 HTTP Cache-Control이 아니다. 서버는 x-nextjs-stale-time으로 클라이언트 라우터에 알린다.

"use cache" 문서는 글이 작성될 때만 바뀌는 글을 max와 태그로 두고, 저장할 때 태그를 지우라고 한다. 문서의 제품 예는 cacheTag('products', 'product-' + id)처럼 목록 태그와 항목 태그를 같이 단다. getCachedPosts와 getCachedPost는 둘 다 태그 "posts" 하나만 단다. postId는 캐시 키의 인자로는 들어가고, 태그 이름에는 들어가지 않는다.

무효화는 actions/postActions.ts가 성공한 뒤에 updateTag("posts")를 호출한다. 작성, 수정, 삭제, 좋아요, 댓글이 모두 같은 태그다. 좋아요 한 번이 목록 캐시와 모든 글 상세 캐시를 함께 만료시킨다.

updateTag는 서버 액션에서만 쓰고, 태그된 캐시를 바로 만료한다. 사용자가 방금 쓴 값을 다음 렌더에서 보게 하는 용도다. revalidateTag는 서버 액션과 Route Handler 모두에서 쓸 수 있고, stale-while-revalidate다. 출처: Revalidating. 이 액션 파일은 revalidateTag, revalidatePath, refresh를 부르지 않는다.

서버 캐시 저장소는 문서 기본값인 인메모리다. 바꾸려면 cacheHandlers를 두라고 한다. 이 설정 파일에는 cacheHandlers가 없다.

"use cache" 안에서는 cookies(), headers(), searchParams를 읽을 수 없다. 호출 스택을 따라가서 도우미가 읽어도 같은 오류다. PostList는 readCurrentUser()를 먼저 부르고, 그 다음 getCachedPosts()를 부른다. 사용자 id는 캐시 함수 인자로 들어가지 않는다. canManage는 캐시 밖에서 post.authorId === viewerId로 계산한다.

connection()은 들어오는 요청을 기다린 뒤에야 이어진 코드가 실행되게 한다. 프리렌더는 그 await에서 멈춘다. 출처: connection. 같은 문서의 Cache Components 항은, 정적 껍데기에서 빼내는 용도에는 io()를 먼저 보고, 실제 사용자 요청을 기다려야 할 때만 connection()을 쓰라고 한다. 이 저장소는 io()를 쓰지 않고 connection()을 세 곳에 둔다.

- lib/currentUser.ts. 이어서 cookies()와 getToken
- lib/readAccessToken.ts. 이어서 cookies()
- component/providers/AuthShell.tsx. 이어서 getServerSession(authOptions)

cookies()만으로도 요청 시점 API다. connection()은 그 앞에 한 번 더 프리렌더를 멈춘다. AuthShell이 루트 레이아웃의 자식 전체를 감싸므로, 그 await 다음에 있는 트리 출력은 요청이 들어온 뒤에 이어진다. 그 안에서 호출된 getCachedPosts는 여전히 "use cache"라서, 태그가 살아 있으면 Prisma를 다시 실

행하지 않고 메모리 값을 반환할 수 있다.

부분 프리페치의 기본은 라우트의 App Shell이다. URL의 `params`나 `searchParams`에 의존하는 `"use cache"`까지 링크 프리페치에 넣으려면 그 `Link`에 `prefetch={true}`를 준다. 이 저장소의 `Link`에는 `prefetch` props가 없다. `getCachedPost(postId)`는 인자로 동적 세그먼트를 받는다.

목록은 글 50개만 읽는다. `listPosts`는 `createdAt` 내림차순 `take: 50`이고, 그 id로 댓글과 좋아요를 `Promise.all`로 한 번씩 더 읽는다. 채우기 한 번의 쿼리 라운드는 3번이다. `getPost`도 3번이다. `findSessionUser`는 `"use cache"` 밖이다.

5. 서버 액션

서버 함수는 `"use server"`로 표시한 비동기 함수다. 폼의 `action`에 넘기면 브라우저는 `FormData`를 POST하고, 그 메서드만 함수를 호출한다. 함수 본문은 직접 POST로도 도달할 수 있어서, 문서가 함수 안에서 인증과 권한을 확인하라고 한다. 출처: Mutating Data.

`postActions.ts`와 `userActions.ts`는 파일 첫 줄에 `"use server"`를 둔다. `PostCard`와 `PostEditor`는 그 파일을 import해서 `action`으로 넘긴다. `FormData.get("body")`는 이름이 `body`인 필드의 값이다. 출처: MDN FormData.

`author()`는 `readCurrentUser()`가 비면 로그인으로 `redirect`한다. 작성 성공 뒤에는 `updateTag("posts")` 다음 `redirect`로 상세에 간다. 수정과 삭제는 작성자가 아니면 service가 `reason: "forbidden"`을 주고 액션은 `/posts`로 보낸다. 좋아요와 댓글은 성공하면 `updateTag`만 하고 `redirect`가 없다.

`redirect`는 예외를 던진다. 그 다음 줄은 실행되지 않는다. 가입 액션은 `try/catch`가 있어서 `unstable_rethrow(error)`를 먼저 부른다. 글 액션은 `redirect`를 `try`로 감싸지 않는다. 좋아요 폼은 다른 URL로 보내지 않고, 태그 만료 뒤 현재 트리가 다시 그려진다.

`experimental.serverActions.bodySizeLimit`은 `"6mb"`다. 설치본 `next@16.4.3`의 `ExperimentalConfig.serverActions.bodySizeLimit`과 같은 자리다. `NextConfig` 최상위에는 이 키가 없다. 가입 4단계가 프로필 파일을 base64 data URL로 만들기 때문에 둔 값이다.

6. 가입은 액션 상태로 단계를 넘긴다

`submitSignup`의 시그니처는 `(prev, formData) => Promise<SignupFormState>`다. React `useActionState`가 이 모양을 받는다. 출처: react.dev useActionState.

`SignupSection`은 `"use client"`다. `useActionState(action, initialSignupState)`로 상태와 액션, pending을 받는다. 제출은 `useOptimistic`으로 단계를 먼저 옮긴 뒤 `formAction(formData)`를 부르고, 둘은 `startTransition` 안에 있다.

단계 값은 쿠키나 `localStorage`에 없다. 화면에 없는 단계는 hidden input으로 다시 보낸다.

보고 있는 단계	hidden으로 다시 보내는 값
1	step. 아이디는 보이는 필드
2	step, userId. 비밀번호는 보이는 필드
3	step, userId, password, passwordConfirm. 이메일은 보이는 필드
4	step, userId, password, passwordConfirm, email. 사진은 파일 필드

postedState가 그 값을 다음 SignupFormState에 넣는다. 그 상태가 hidden input의 value가 된다. 비밀번호도 이 상태로 브라우저에 남는다.

SignupSubmitButton은 useFormStatus()를 쓴다. 이 훅은 자신을 감싼 폼의 제출 상태를 읽는다. 버튼 name="intent"의 값은 next, back, skip이다. pending 중에는 status.data.get("intent")가 그 버튼과 같을 때만 그 버튼의 pending 문구를 보여 준다.

액션은 단계마다 service로 아이디, 비밀번호, 이메일을 확인하고, 4단계에서 userService.create 뒤에 redirect("/login")을 한다. 관심사 배열은 []로 넘긴다. 관심사 선택 폼 파일은 현재 트리에 없다.

7. 로그인, 세션, Proxy

LoginSection은 "use client"이고 useSearchParams().get("callbackUrl")을 safeCallbackPath에 넣는다. 이 훅은 가장 가까운 Suspense가 필요하다. 로그인 페이지가 fallback={null}로 감싼다. 출처: useSearchParams.

LoginForm은 preventDefault 뒤 signIn("credentials", { email, password, redirect: true, callbackUrl })를 부른다. 서버 액션이 아니다. "use client" 모듈이 로드된 뒤 호출되고, NextAuth 엔드포인트로 이동한다. 구글과 카카오 버튼도 signIn("google"), signIn("kakao")다.

rememberId는 useState(false)와 체크박스 checked만 있다. 제출 때 읽지 않고, localStorage나 sessionStorage에 쓰지 않는다.

app/api/auth/[...nextauth]/route.ts는 NextAuth(authOptions) 핸들러를 GET과 POST로 export한다. 세션은 strategy: "jwt", maxAge는 2시간이다. pages.signIn은 /login이다.

공급자	콜백에서 하는 일
credentials	userService.login이 성공하면 authorize가 accessToken을 돌려주고, jwt 콜백이 token.accessToken에 넣는다
kakao, google	userService.linkSocialAccount로 연결한 뒤 signAccessToken으로 새 JWT를 만들어 token.accessToken에 넣는다

lib/accessJwt.ts는 jose의 SignJWT다. 보호 헤더 alg: "HS256", sub는 사용자 id, email과 name은 페이로드, 만료는 2시간, 서명은 NEXTAUTH_SECRET이다. verifyAccessToken은 jwtVerify로 HS256만 허용하고, sub가 24자리 hex가 아니면 null이다.

AuthShell은 세션의 accessToken을 다시 확인한 뒤에만 AuthProvider에 넘긴다. isLoggedIn은 Boolean(accessToken)이다. LoginStatus는 그 값으로 로그인 링크와 signOut({ callbackUrl: "/" })을 가른다.

proxy.ts는 app과 같은 단에 있고 함수 이름은 proxy다. Next.js는 middleware 파일 규약을 proxy로 바꿨다. Proxy는 라우트 렌더 전에 실행되고, 리다이렉트와 쿠키, 헤더로만 앱에 정보를 넘긴다. 출처: proxy 파일 규약.

조건	응답
경로가 /login 또는 /signup이고 로그인됨	safeCallbackPath(callbackUrl)로 302. 기본은 /
/cart, /order이거나 /postwrite로 시작하고 로그인 안 됨	/login?callbackUrl= 현재 경로와 쿼리
그 외	NextResponse.next()

safeCallbackPath는 /로 시작하지 않거나 //로 시작하면 /를 반환한다. 목적지가 /login이나 /signup이어도 /다.

config.matcher는 api, _next/static, _next/image, favicon.ico, sitemap.xml, robots.txt를 제외한다. /posts와 /postdetail은 인증 목록에 없다. 읽기는 Proxy가 막지 않는다. 쓰기는 /postwrite에서 Proxy가 막고, 액션의 author()가 한 번 더 막는다. 수정과 삭제는 service가 authorId를 비교한다.

app/api/auth/login/route.ts와 app/api/user/route.ts는 JSON API다. CORS는 Access-Control-Allow-Origin: *이고 OPTIONS는 204다. 로그인은 Zod 검사 뒤 200 또는 401을 주고, NextAuth 쿠키를 심지 않고 accessToken 문자열을 JSON으로 돌려준다. 넥스트 화면의 LoginForm은 이 라우트를 fetch하지 않는다.

8. 글 본문

PostEditor는 클라이언트 컴포넌트다. Quill 생성은 effect 안의 await import("quill")이다. 툴바는 헤딩, 굵게, 기울임, 밑줄, 취소선, 목록, 인용, 링크, 서식 지우기다. 이미지 버튼은 없다. 변경 때마다 getSemanticHTML()이 hidden input name="body"에 들어간다. 보이는 글자가 없으면 제출 버튼이 disabled다. cleanup은 .ql-toolbar를 지우고 host의 innerHTML을 비운다.

preparePostBody는 sanitizePostHtml로 허용 태그만 남긴 뒤, 태그를 뺀 글자 수가 1 이상 5000 이하인지 본다. 허용 태그는 p, br, strong, em, u, s, a, ol, ul, li, h1, h2, h3, blockquote, span이다. a의 스킴은 http, https, mailto다. 댓글은 trim과 300자 제한만 있고, React 텍스트 자식으로 그린다.

상세에서 full이고 본문이 <로 시작하면 dangerouslySetInnerHTML로 넣는다. React는 평범한 텍스트 자식을 이스케이프하고, 이 prop는 그 이스케이프를 건너뛰고 HTML로 파싱한다. 출처: React dangerouslySetInnerHTML. 저장 전에 서버가 허용 목록으로 걸러 둔 문자열만 이 경로로 온다. 목록 요약은 정규식으로 태그를 벗겨 140자로 자른 텍스트다.

Prisma 모델은 Post, Comment, Like다. Like는 @@unique([postId, userId])라서 있으면 지우고 없으면 만든다. deletePost는 댓글, 좋아요, 글 순으로 지운다. 트랜잭션 호출은 없다.

lib/prisma.ts는 APP_ENV가 local 또는 develop일 때 해당 URL을 DATABASE_URL에 넣고, globalThis에 PrismaClient 하나를 보관한다. 출처: Prisma, instantiate Prisma Client.

9. 설정이 문서와 맞는 곳

설정 또는 호출	이 저장소	공식 문서
`cacheComponents`, `partialPrefetching`	`next.config.ts`에 둘 다 `true`	시작 문서의 예와 같다
`"use cache"` + `cacheLife("max")` + `cacheTag`	`lib/postCache.ts`	긴 수명과 태그를 같이 두는 예와 같다
`updateTag`를 서버 액션에서 호출	글 변경 다섯 함수	read-your-own-writes 용도와 같다
쿠키를 캐시 함수 밖에서 읽음	`PostList`, `PostDetail`, `PostForm`	`use cache` 제한과 같다
`params`를 Promise로 await	피드, 상품, 블로그, 글	Next.js 15 비동기 request API와 같다
서버 함수 안에서 로그인과 작성자 확인	`author()`, `updatePost`, `deletePost`	직접 POST에도 권한을 확인하라는 문장과 같다
Proxy 파일 이름과 matcher 제외 규칙	`proxy.ts`	`middleware`에서 바뀐 규약, 정적 파일 제외 예와 같다
`serverActions.bodySizeLimit`	`experimental` 안 `"6mb"`	이 버전의 타입은 `ExperimentalConfig`에만 있다
항목별 캐시 태그	태그 문자열은 `"posts"` 하나	문서 예는 목록 태그와 `product-${id}`를 같이 단다
`Link`의 `prefetch={true}`	없음	URL 인자에 의존하는 캐시를 프리페치에 넣으려면 이 prop를 요구한다
`io()`	없음	껍데기에서 빼낼 때 문서가 `connection()`보다 먼저 권한다
`connection()` 위치	루트 `AuthShell`과 사용자 조회	문서의 예는 페이지 안에서 요청마다 달라지는 값 앞이다
아이디 기억 체크박스	`rememberId` state만 존재	저장 API를 호출하는 코드는 없다
`client-login`	콘솔 출력	인증 라우트를 호출하지 않는다

이미지 설정은 `images.remotePatterns`로 `i.pinimg.com`과 `www.lgcns.com`의 `https`만 허용한다. `experimental.agentFeedback`은 `true`다. 글 조회나 인증 동작에는 쓰이지 않는다.

CSS는 Tailwind 4다. `turbopack.rules`가 `*.css`에 `@tailwindcss/turbopack` 로더를 붙인다. 조건은 `{ not: "foreign" }`이다. 이 로더는 스타일 파이프라인이고, 페이지의 `<script>` 로딩과는 별개다.

10. 글 하나가 저장될 때까지

1. 브라우저가 `/postwrite`를 연다. `proxy.ts`가 먼저 실행된다. `accessToken`이 없으면 `/login?callbackUrl=/postwrite`로 보낸다.
2. 로그인 페이지의 클라이언트 모듈이 하이드레이션된 뒤 `signIn`이 NextAuth `POST`로 이동한다. `jwt` 콜백이 `token.accessToken`에 `jose` JWT를 넣는다.

3. 글쓰기 페이지가 렌더된다. PostForm은 Suspense 안에서 readCurrentUser로 다시 확인하고, 없으면 로그인으로 redirect한다.
4. PostEditor가 마운트되면 import("quill")이 청크를 받는다. 그 전 HTML에는 에디터 인스턴스가 없다.
5. 제출은 폼 action={createPost}다. POST의 FormData에 hidden body가 실린다. 페이지에 손쓴 <script>는 없다.
6. 서버 함수가 사용자를 확인하고 postService.createPost가 HTML을 정제한 뒤 prisma.post.create를 한다.
7. updateTag("posts")가 태그 "posts"를 만료시킨다. redirect가 /postdetail/[id]로 보낸다.
8. 상세 페이지는 getCachedPost(postId)를 부른다. 태그가 방금 지워졌으므로 이 호출은 Prisma 3라운드로 다시 채운다. 같은 태그가 살아있는 동안의 다음 읽기는 그 메모리 값을 쓴다.